JC Contract Integration — API Design Document

> **Repository:** `aptos-labs/jc-contract-integration` **Language / Framework:** Go (Gin / Echo) + Aptos Go SDK **Last Updated:** 2026-02-27

Table of Contents

1. Executive Summary
2. High-Level Architecture
3. System Context Diagram
4. Component Architecture
5. API Endpoint Design
6. Data Models & Schemas
7. Request / Response Lifecycle
8. Authentication & Authorization
9. Error Handling Strategy
10. Database Design
11. Sequence Diagrams
12. Design Decisions & Tradeoffs
13. Future Expansions
14. Future Testing & Integration Plans
15. Appendix

---

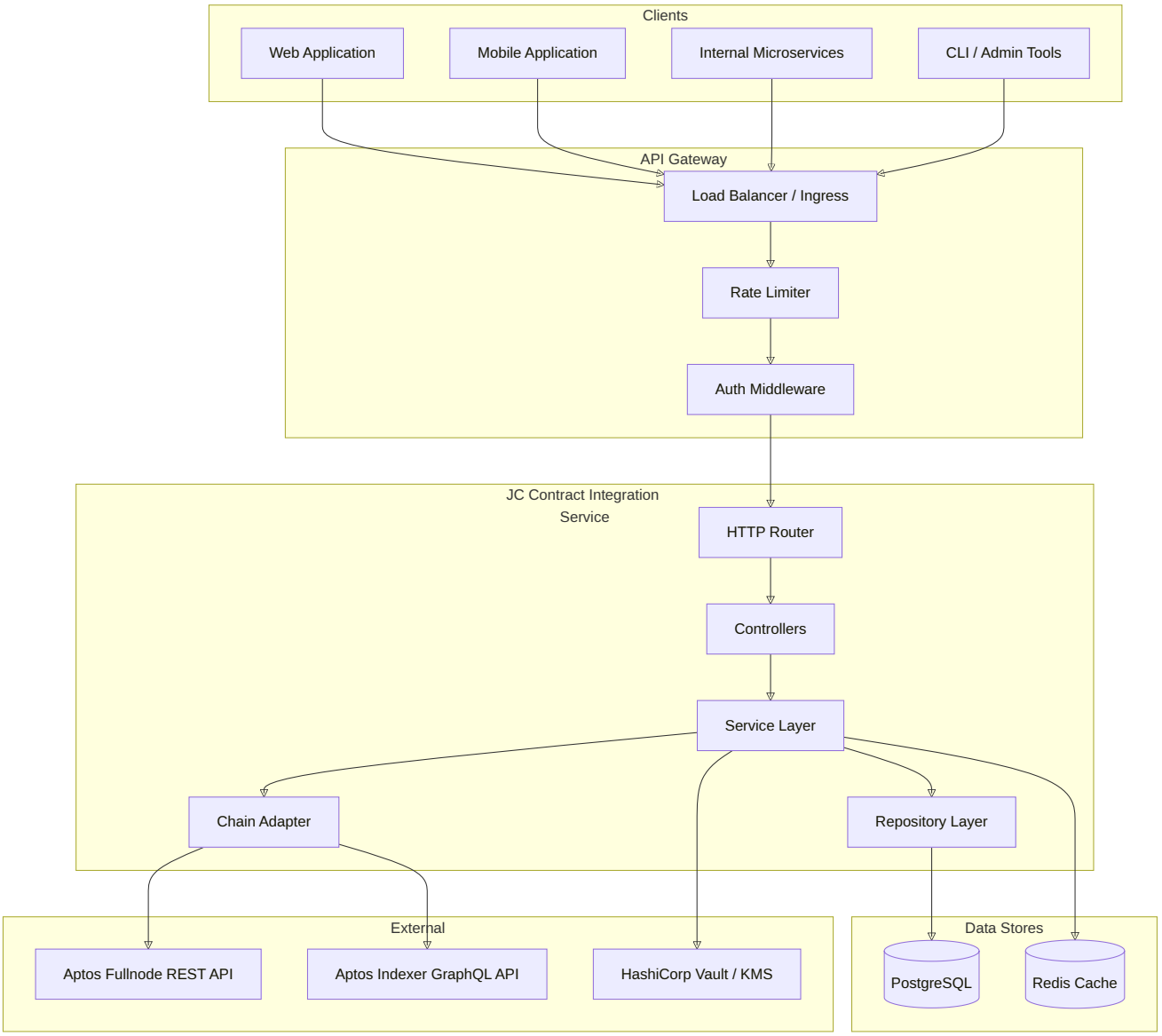
1. Executive Summary

The **JC Contract Integration** service is a Go-based HTTP API that acts as a middleware layer between client applications and the Aptos blockchain. It abstracts the complexity of direct blockchain interaction — transaction construction, BCS serialization, gas estimation, sequence-number management, and event indexing — behind a clean, versioned REST API.

Goals

Goal	Description
Abstraction	Hide Move/BCS complexity from API consumers
Reliability	Guarantee transaction delivery with retries and idempotency
Observability	Full request tracing, metrics, and structured logging
Security	Key management via HSM/vault; scoped API-key access
Scalability	Stateless compute with horizontal scaling behind a load balancer

2. High-Level Architecture

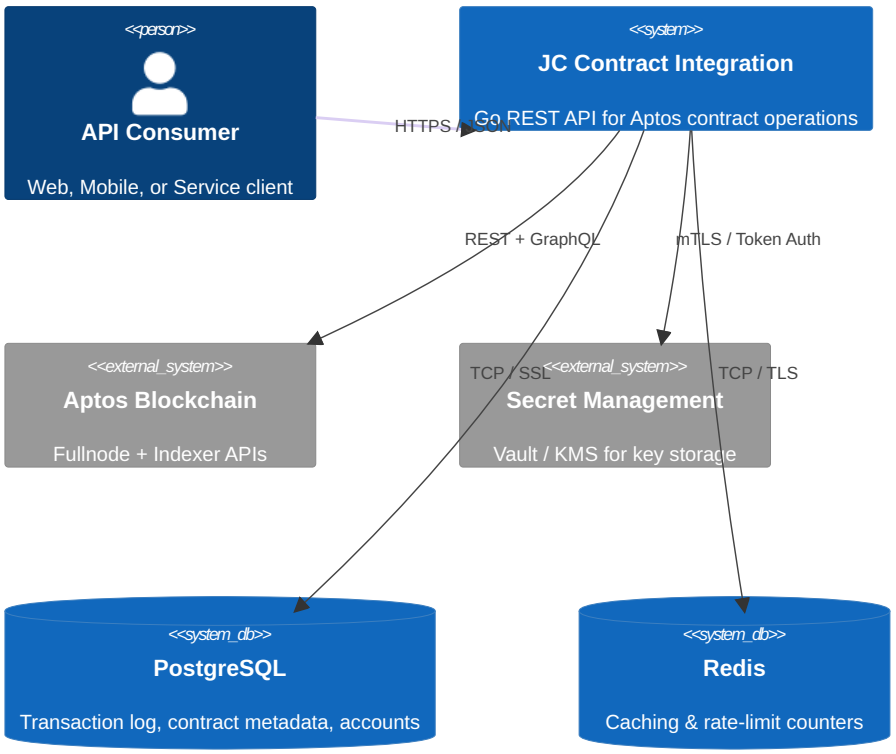


Layer Responsibilities

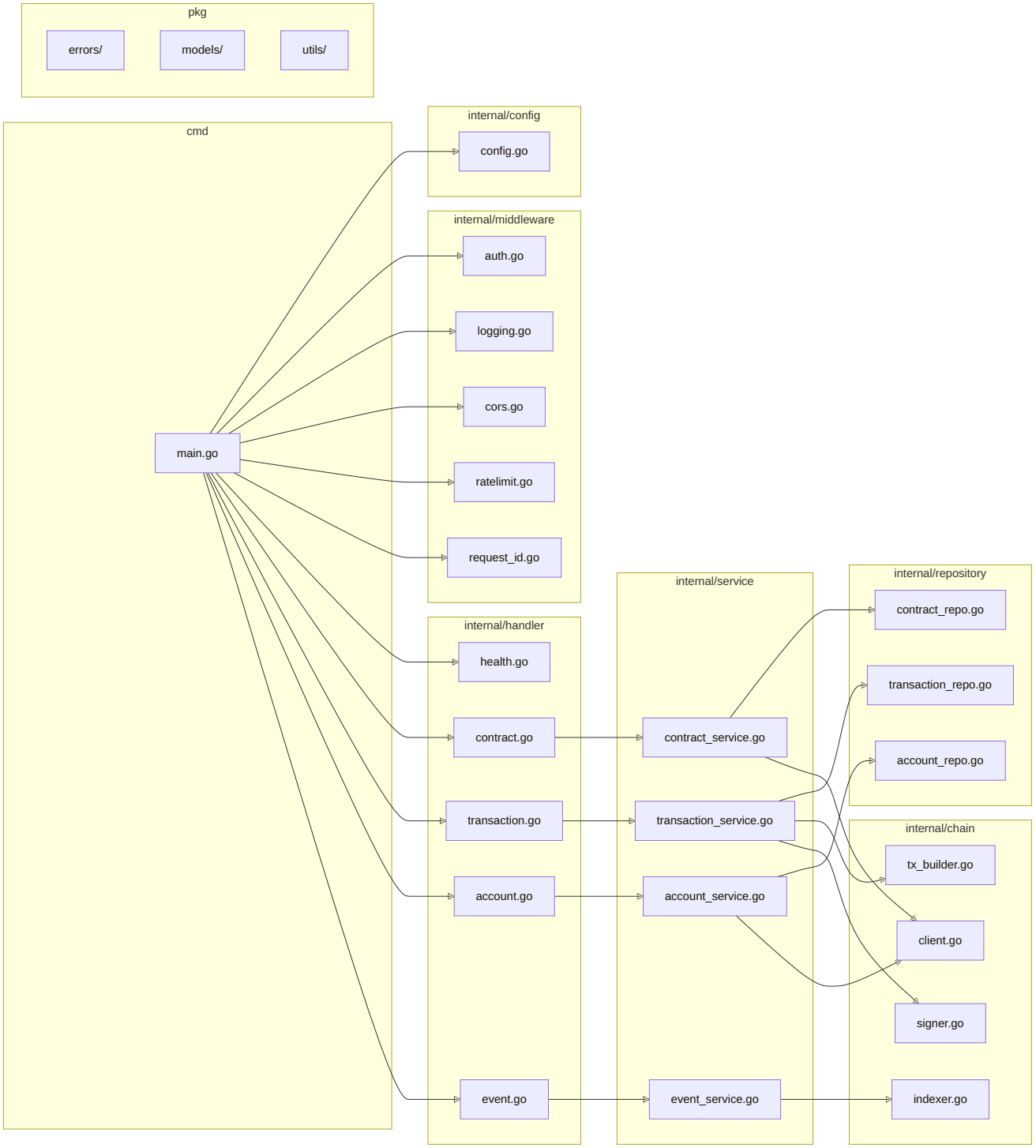
Layer	Responsibility
HTTP Router	Route matching, versioning (`/v1/ ...`), request parsing
Controllers	Input validation, request ↔ response mapping
Service Layer	Business logic, transaction orchestration, caching strategy
Repository Layer	Database CRUD, query building, migrations
Chain Adapter	Aptos Go SDK wrapper — transaction build, sign, submit, wait

3. System Context Diagram

System Context — JC Contract Integration



4. Component Architecture



Directory Layout

```
jc-contract-integration/
├── cmd/
│   └── server/
│       └── main.go              # Entrypoint
├── internal/
│   ├── config/                  # Env-based configuration
│   ├── middleware/               # HTTP middleware stack
│   ├── handler/                 # HTTP handlers (controllers)
│   ├── service/                 # Business logic
│   ├── repository/              # Data access
│   └── chain/                   # Aptos blockchain adapter
├── pkg/
│   ├── models/                  # Shared domain models
│   ├── errors/                  # Custom error types
│   └── utils/                   # Helpers (pagination, validation)
├── migrations/                  # SQL migration files
├── docs/                        # This document & OpenAPI spec
├── scripts/                     # Dev tooling, seed data
├── .env.example
├── Dockerfile
├── docker-compose.yml
├── Makefile
├── go.mod
└── go.sum
```

---

5. API Endpoint Design

All endpoints are prefixed with `/api/v1`. Responses follow a consistent envelope.

5.1 Health & Meta

Method	Path	Description
`GET`	`/healthz`	Liveness probe
`GET`	`/readyz`	Readiness probe (DB + chain connectivity)
`GET`	`/api/v1/info`	Service version, chain ID, connected network

5.2 Contracts

Method	Path	Description
`POST`	`/api/v1/contracts`	Register a contract (module address + name)

Method	Path	Description
`GET`	`/api/v1/contracts`	List registered contracts (paginated)
`GET`	`/api/v1/contracts/:id`	Get contract details + ABI
`DELETE`	`/api/v1/contracts/:id`	Deregister a contract
`POST`	`/api/v1/contracts/:id/call`	Call a view function (read-only)
`POST`	`/api/v1/contracts/:id/execute`	Execute an entry function (state-changing)
`POST`	`/api/v1/contracts/:id/simulate`	Simulate a transaction before submission

5.3 Transactions

Method	Path	Description
`GET`	`/api/v1/transactions`	List tracked transactions (paginated, filterable)
`GET`	`/api/v1/transactions/:hash`	Get transaction status & details
`POST`	`/api/v1/transactions/submit`	Submit a raw signed transaction
`POST`	`/api/v1/transactions/batch`	Submit a batch of transactions

5.4 Accounts

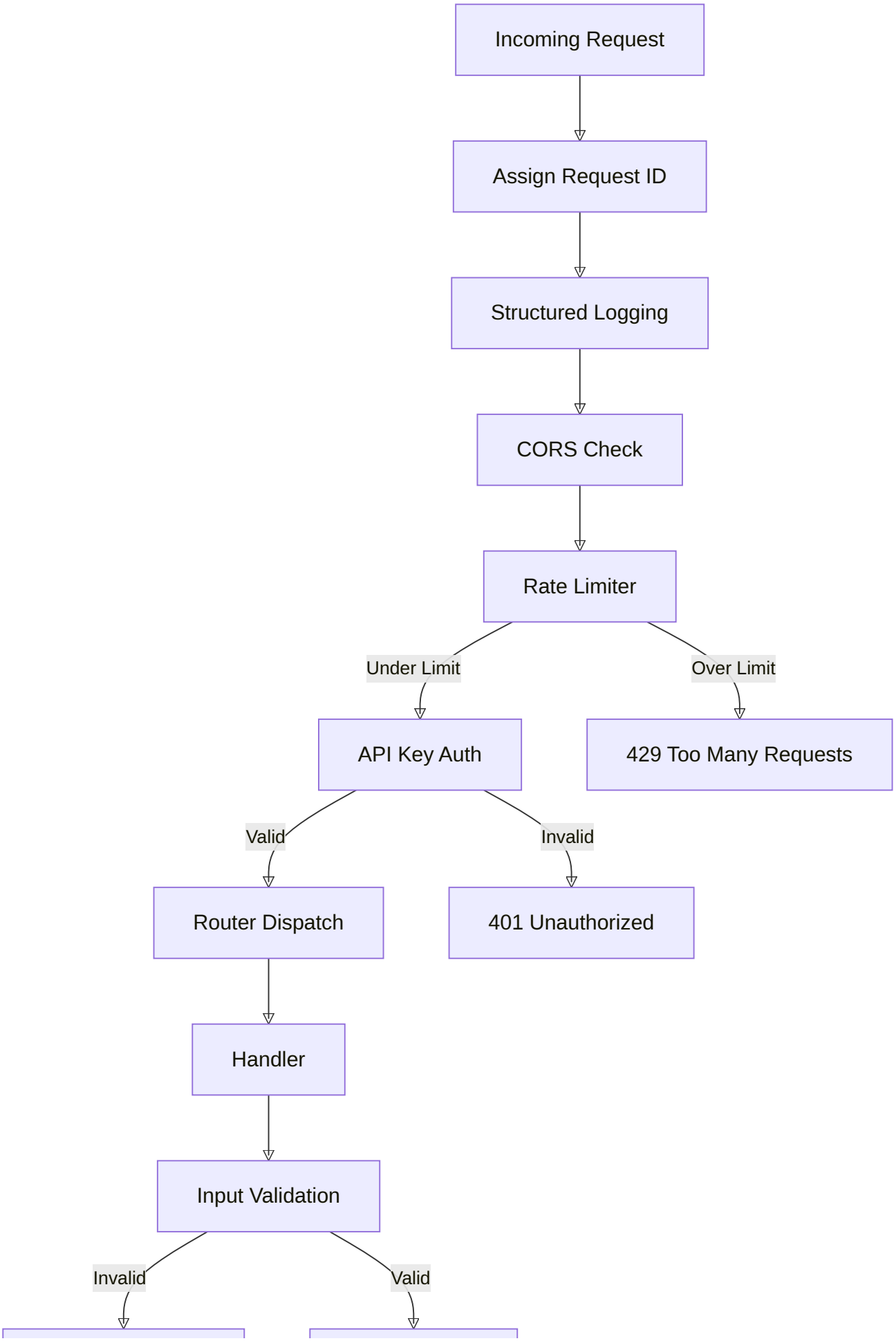
Method	Path	Description
`POST`	`/api/v1/accounts`	Create / register a managed account
`GET`	`/api/v1/accounts/:address`	Get account info (sequence number, resources)
`GET`	`/api/v1/accounts/:address/resources`	List account resources
`GET`	`/api/v1/accounts/:address/modules`	List published modules
`GET`	`/api/v1/accounts/:address/balance`	Get APT balance

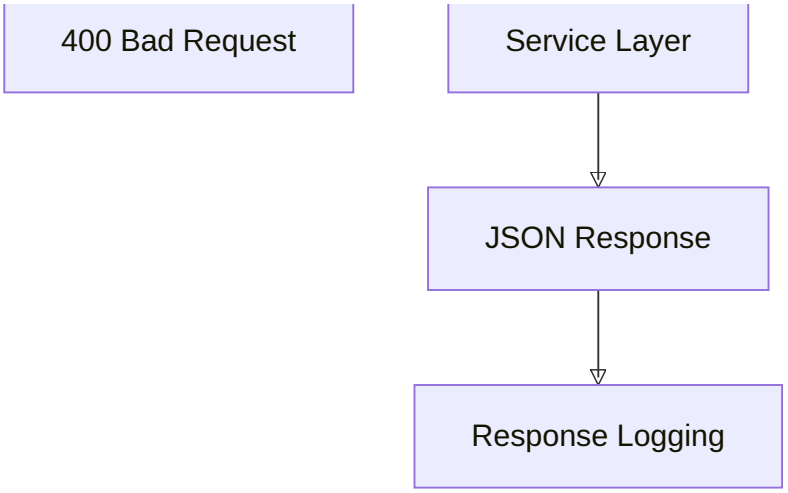
5.5 Events

Method	Path	Description
`GET`	`/api/v1/events`	Query events (by contract, type, time range)
`POST`	`/api/v1/events/subscribe`	Create a webhook subscription for events

Method	Path	Description
`DELETE`	`/api/v1/events/subscribe/:id`	Remove a webhook subscription
`GET`	`/api/v1/events/subscribe`	List active subscriptions

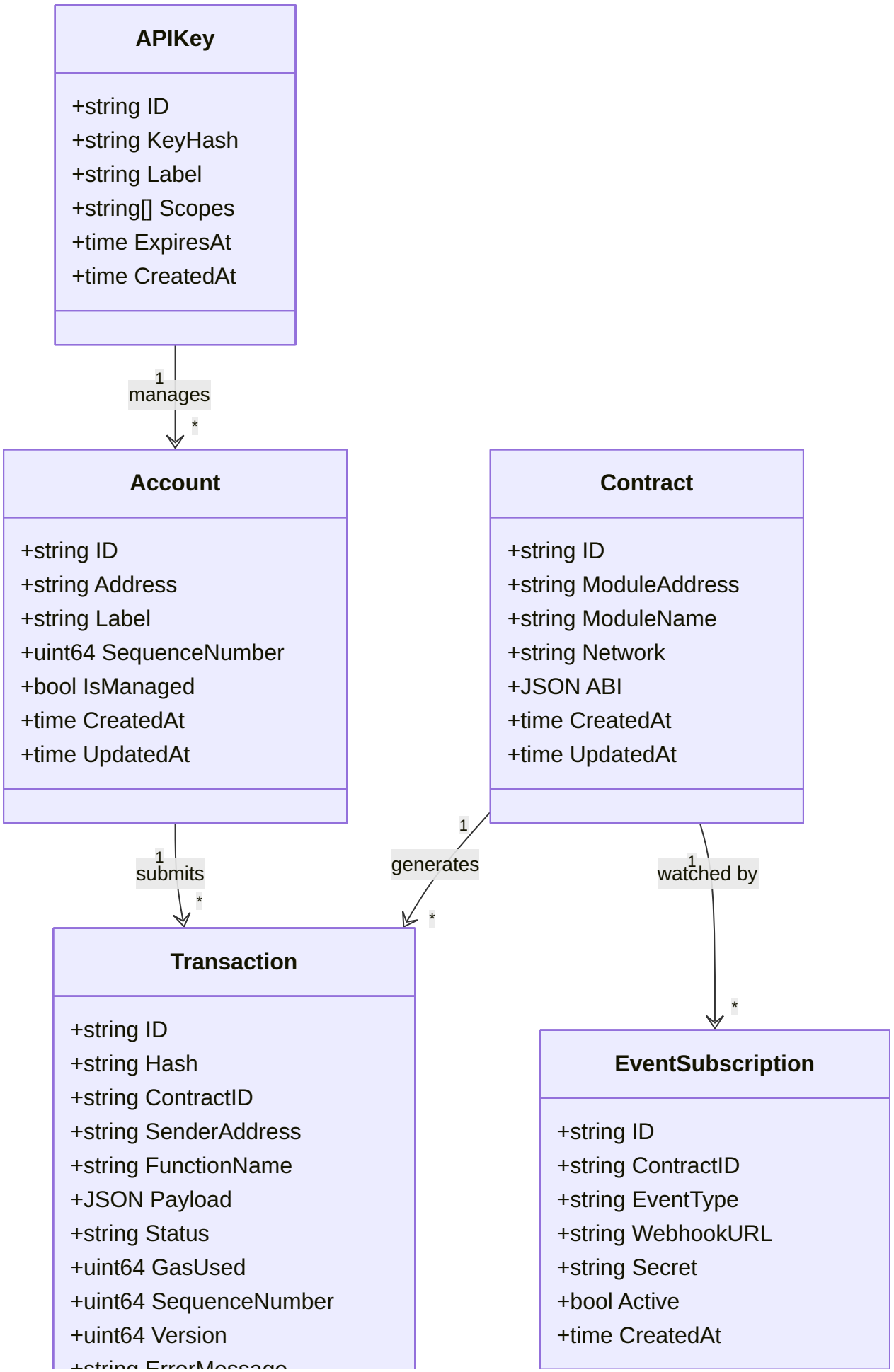
5.6 Endpoint Flow Diagram

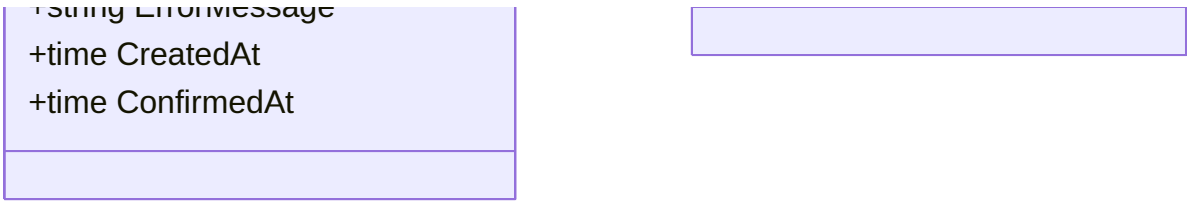

Incoming Request
Assign Request ID
Structured Logging
CORS Check
Rate Limiter
Under Limit
Over Limit
API Key Auth
429 Too Many Requests
Valid
Invalid
Router Dispatch
401 Unauthorized
Handler
Input Validation
Invalid
Valid


400 Bad Request
Service Layer
JSON Response
Response Logging

6. Data Models & Schemas

6.1 Core Domain Models


APIKey
+string ID
+string KeyHash
+string Label
+string[] Scopes
+time ExpiresAt
+time CreatedAt
1
manages
*
Account
+string ID
+string Address
+string Label
+uint64 SequenceNumber
+bool IsManaged
+time CreatedAt
+time UpdatedAt
Contract
+string ID
+string ModuleAddress
+string ModuleName
+string Network
+JSON ABI
+time CreatedAt
+time UpdatedAt
1
submits
*
1
generates
*
1
watched by
*
Transaction
+string ID
+string Hash
+string ContractID
+string SenderAddress
+string FunctionName
+JSON Payload
+string Status
+uint64 GasUsed
+uint64 SequenceNumber
+uint64 Version
EventSubscription
+string ID
+string ContractID
+string EventType
+string WebhookURL
+string Secret
+bool Active
+time CreatedAt



6.2 Response Envelope

All API responses follow this structure:

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "request_id": "req_abc123",
    "timestamp": "2026-02-27T12:00:00Z"
  },
  "pagination": {
    "page": 1,
    "per_page": 25,
    "total": 142,
    "total_pages": 6
  }
}
```

Error responses:

```
{
  "success": false,
  "error": {
    "code": "INVALID_PAYLOAD",
    "message": "Field 'module_address' is required",
    "details": { ... }
  },
  "meta": {
    "request_id": "req_abc123",
    "timestamp": "2026-02-27T12:00:00Z"
  }
}
```

6.3 Key Request / Response Shapes

POST `/api/v1/contracts/:id/execute`

Request:

```
{
  "function_name": "transfer",
  "type_arguments": ["0x1::aptos_coin::AptosCoin"],
  "arguments": ["0xrecipient...", "1000000"],
  "sender_address": "0xsender...",
  "max_gas_amount": 10000,
  "gas_unit_price": 100,
  "idempotency_key": "ik_unique_transfer_001"
}
```

Response (202 Accepted):

```
{
  "success": true,
  "data": {
    "transaction_id": "txn_abc123",
    "hash": "0xabc...def",
    "status": "pending",
    "submitted_at": "2026-02-27T12:00:00Z"
  },
  "meta": {
    "request_id": "req_xyz789",
    "timestamp": "2026-02-27T12:00:00Z"
  }
}
```

POST `/api/v1/contracts/:id/call` (View Function)

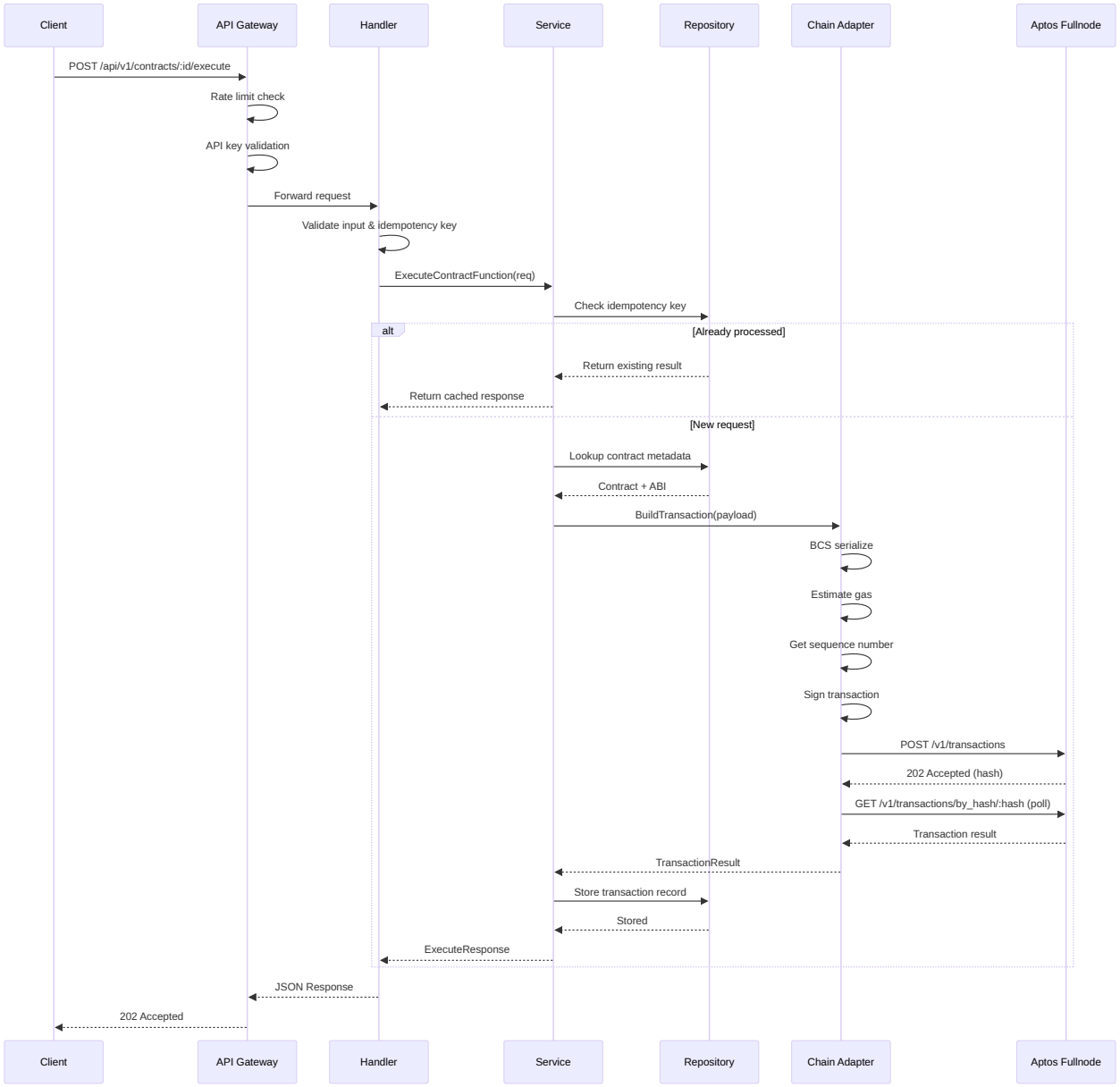
Request:

```
{
  "function_name": "get_balance",
  "type_arguments": ["0x1::aptos_coin::AptosCoin"],
  "arguments": ["0xaccount..."]
}
```

Response (200 OK):

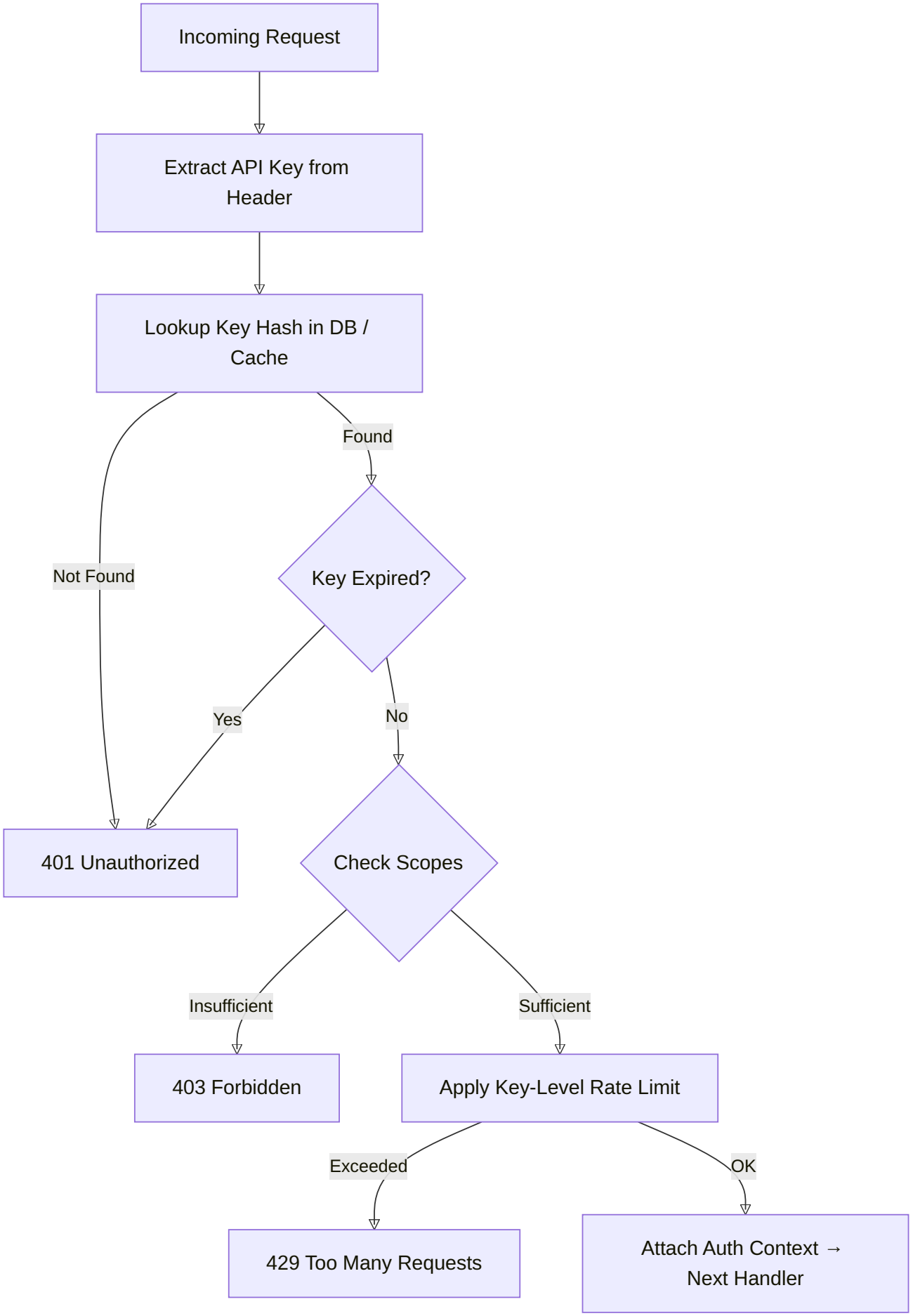
```
{
  "success": true,
  "data": {
    "result": ["50000000"]
  },
  "meta": {
    "request_id": "req_xyz790",
    "timestamp": "2026-02-27T12:00:00Z"
  }
}
```

7. Request / Response Lifecycle



8. Authentication & Authorization

Auth Flow


Incoming Request
Extract API Key from Header
Lookup Key Hash in DB / Cache
Found
Not Found
Key Expired?
Yes
No
401 Unauthorized
Check Scopes
Insufficient
Sufficient
403 Forbidden
Apply Key-Level Rate Limit
Exceeded
OK
429 Too Many Requests
Attach Auth Context → Next Handler

Scope Model

Scope	Grants Access To
`contracts:read`	GET contract endpoints
`contracts:write`	POST/DELETE contract endpoints
`contracts:execute`	Execute and simulate transactions
`transactions:read`	GET transaction endpoints
`transactions:submit`	Submit raw transactions
`accounts:read`	GET account endpoints
`accounts:manage`	Create / manage accounts
`events:read`	GET event endpoints
`events:subscribe`	Manage webhook subscriptions
`admin`	Full access including key management

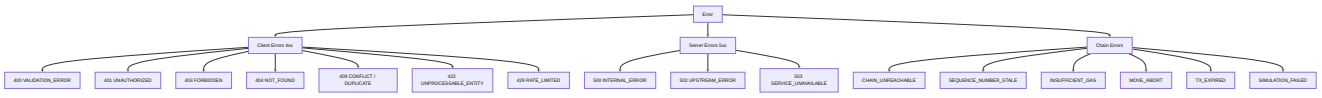
Key Management

- API keys are generated as 256-bit random tokens, prefixed with `jcint_`
- Only the **SHA-256 hash** of the key is stored in the database
- Keys support optional expiration dates
- Keys are scoped to specific permission sets
- Rotation: new key issued → old key enters grace period → old key revoked

---

9. Error Handling Strategy

Error Code Taxonomy



Error Response Contract

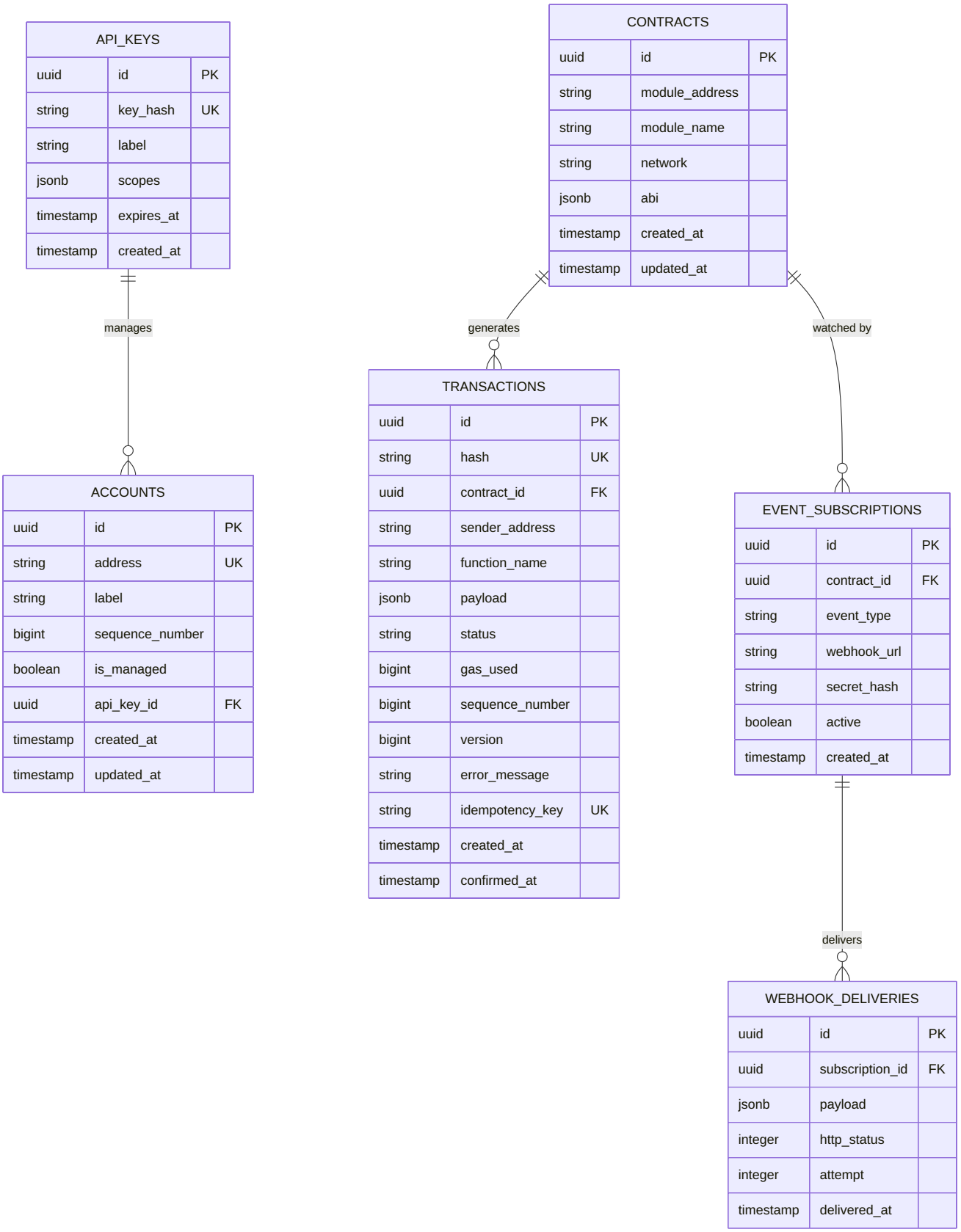
```
{
  "success": false,
  "error": {
    "code": "MOVE_ABORT",
    "message": "Transaction aborted by Move module",
    "details": {
      "abort_code": 5,
      "module": "0x1::coin",
      "function": "transfer",
      "description": "Insufficient balance"
    }
  },
  "meta": {
    "request_id": "req_abc123",
    "timestamp": "2026-02-27T12:00:01Z"
  }
}
```

Retry Policy (Chain Adapter)

Error Category	Retryable	Strategy
Network timeout	Yes	Exponential backoff (1s, 2s, 4s) — max 3
5xx from Aptos node	Yes	Exponential backoff — max 3
Stale sequence number	Yes	Re-fetch sequence number, retry once
Move abort	No	Return error immediately
Invalid payload	No	Return 400 immediately

10. Database Design

Entity-Relationship Diagram



Indexes

Table	Index	Purpose
`transactions`	`idx_tx_hash` (UNIQUE)	Fast lookup by on-chain hash
`transactions`	`idx_tx_idempotency` (UNIQUE)	Idempotency deduplication
`transactions`	`idx_tx_contract_status`	Filter by contract + status
`transactions`	`idx_tx_sender_created`	Sender history, ordered
`contracts`	`idx_contract_addr_name` (UNIQUE)	Dedup registration
`accounts`	`idx_account_address` (UNIQUE)	Address lookup
`api_keys`	`idx_key_hash` (UNIQUE)	Auth lookup

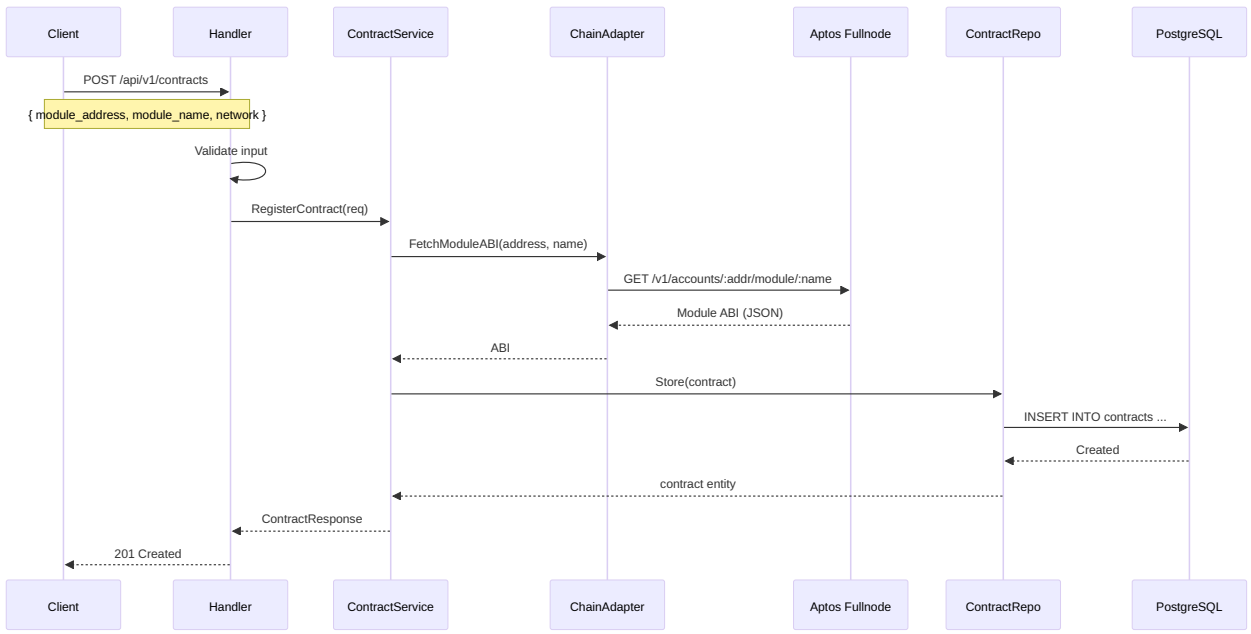
Migration Strategy

- Migrations managed by golang-migrate
- Numbered files: `000001_create_contracts.up.sql` / `000001_create_contracts.down.sql`
- Applied automatically on startup in dev; manually via CI in production

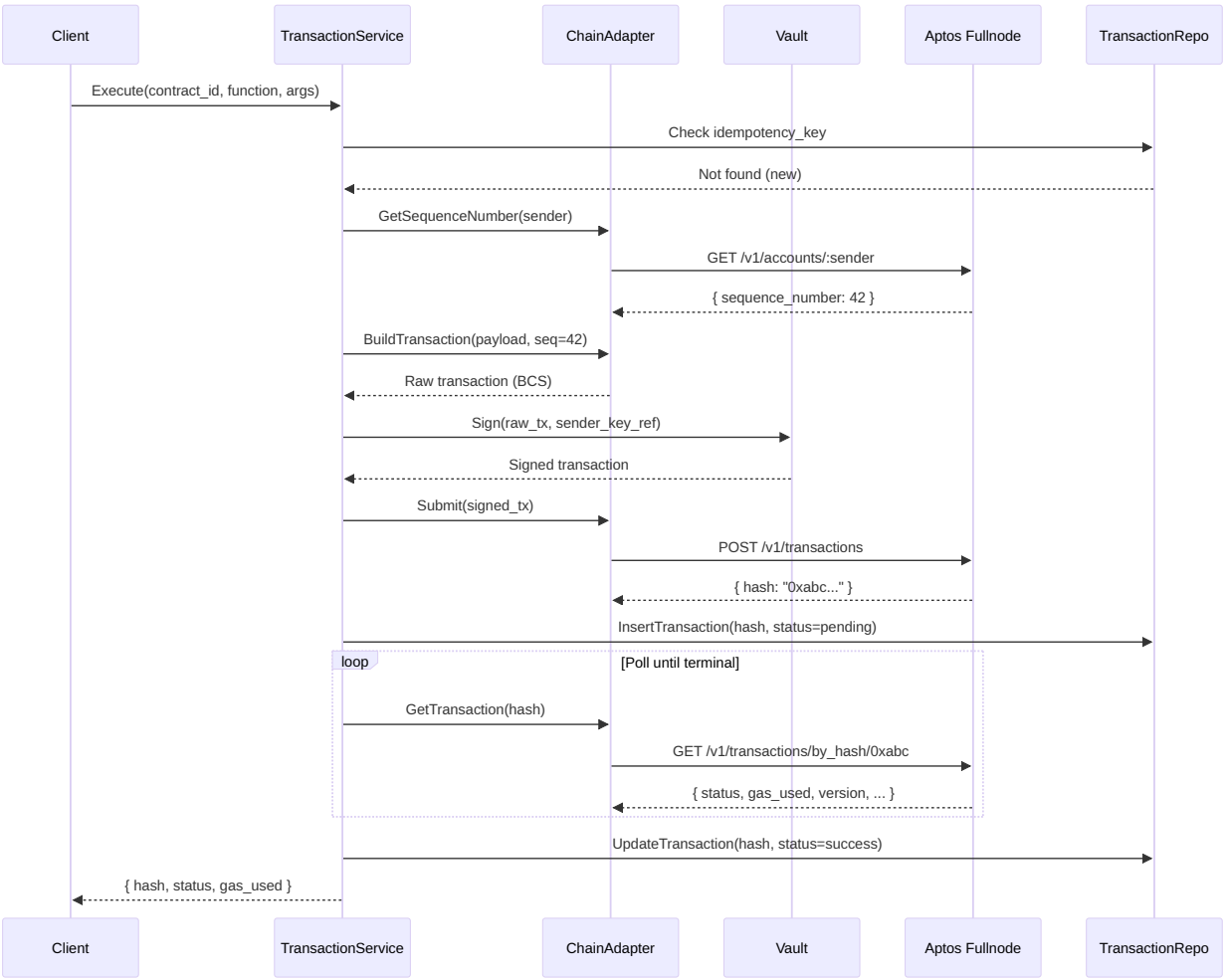
---

11. Sequence Diagrams

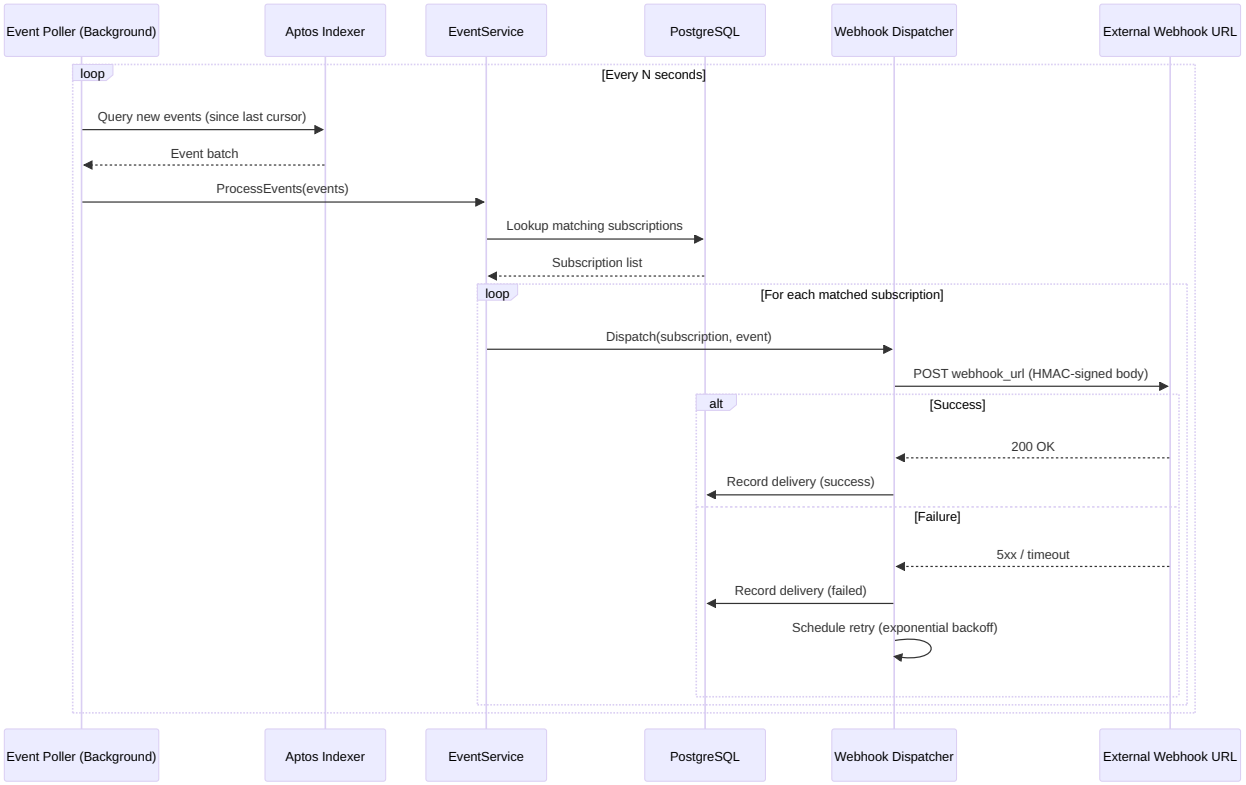
11.1 Contract Registration



11.2 Transaction Execution (State-Changing)



11.3 Event Webhook Delivery



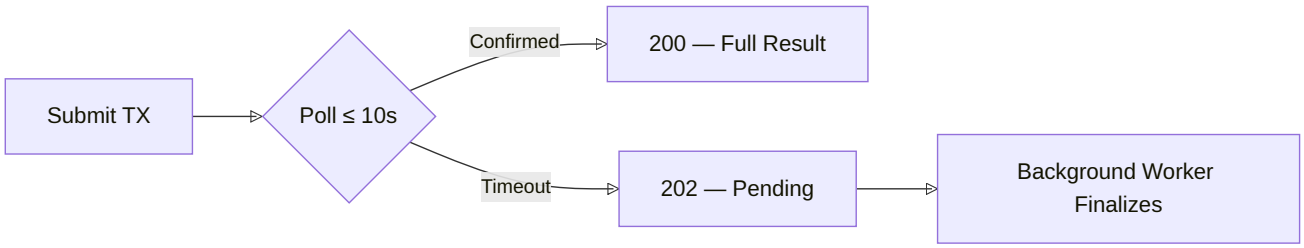
12. Design Decisions & Tradeoffs

Decision 1: Go as the Implementation Language

Aspect	Detail
Decision	Use Go instead of TypeScript/Rust
Rationale	Strong concurrency model (goroutines) for parallel chain interactions; first-class Aptos Go SDK support; fast compile times; simple deployment as a single binary
Tradeoff	Smaller Move/Aptos ecosystem compared to TypeScript. The official TypeScript SDK receives updates first. Go SDK may lag behind on new features
Mitigation	Pin SDK version; contribute upstream patches when needed

Decision 2: Synchronous Transaction Submission with Polling

Aspect	Detail
Decision	The `/execute` endpoint submits the transaction and polls for confirmation before returning (up to a timeout), returning `202 Accepted` immediately if confirmation takes too long
Rationale	Simpler client integration — callers get a terminal status in most cases without implementing their own polling
Tradeoff	Holds an HTTP connection open longer; under load this consumes server resources
Mitigation	Configurable poll timeout (default 10s); background worker finalizes status for timed-out requests; clients can opt for fire-and-forget mode via `?async=true` query param



Decision 3: Server-Side Signing via Vault

Aspect	Detail
Decision	Private keys are stored in HashiCorp Vault (or cloud KMS). The service signs transactions server-side
Rationale	Centralizes key management; avoids key exposure in client applications; supports HSM-backed keys for production
Tradeoff	Introduces Vault as a critical dependency; increases latency for each sign operation (~5-15ms); requires trust in the service operator
Alternative Considered	Client-side signing where the service only relays pre-signed transactions. Decided to support *both* modes: managed accounts (server-signs) and relay mode (client-signs, service submits)

Decision 4: PostgreSQL for Transaction Logging

Aspect	Detail
Decision	Use PostgreSQL (not a NoSQL store) for transaction records, contracts, and accounts

Aspect	Detail
Rationale	Strong consistency guarantees for idempotency checks; rich query support for filtering transactions; JSONB for flexible ABI/payload storage; battle-tested in production
Tradeoff	Schema rigidity requires migrations; horizontal scaling is harder than with DynamoDB/Cassandra
Mitigation	Partition `transactions` table by `created_at` for time-range queries; read replicas for GET-heavy endpoints

Decision 5: Idempotency Keys for Mutation Endpoints

Aspect	Detail
Decision	All state-changing endpoints accept an `idempotency_key` header/field. Duplicate keys return the original response
Rationale	Blockchain transactions are irreversible. Network retries or client bugs must not double-submit
Tradeoff	Adds storage overhead (unique index on idempotency key); requires TTL-based cleanup for old keys
Implementation	Keys stored with a 72-hour TTL. After expiry, the same key can be reused

Decision 6: Rate Limiting Strategy

Aspect	Detail
Decision	Token-bucket rate limiting per API key, backed by Redis
Rationale	Protects the service and downstream Aptos nodes from abuse; fair resource allocation across tenants
Tradeoff	Adds Redis as a runtime dependency; distributed rate limiting has slight inaccuracy under race conditions
Defaults	100 requests/minute for read endpoints; 20 requests/minute for execute/submit endpoints

Decision 7: Versioned API with `/v1` Prefix

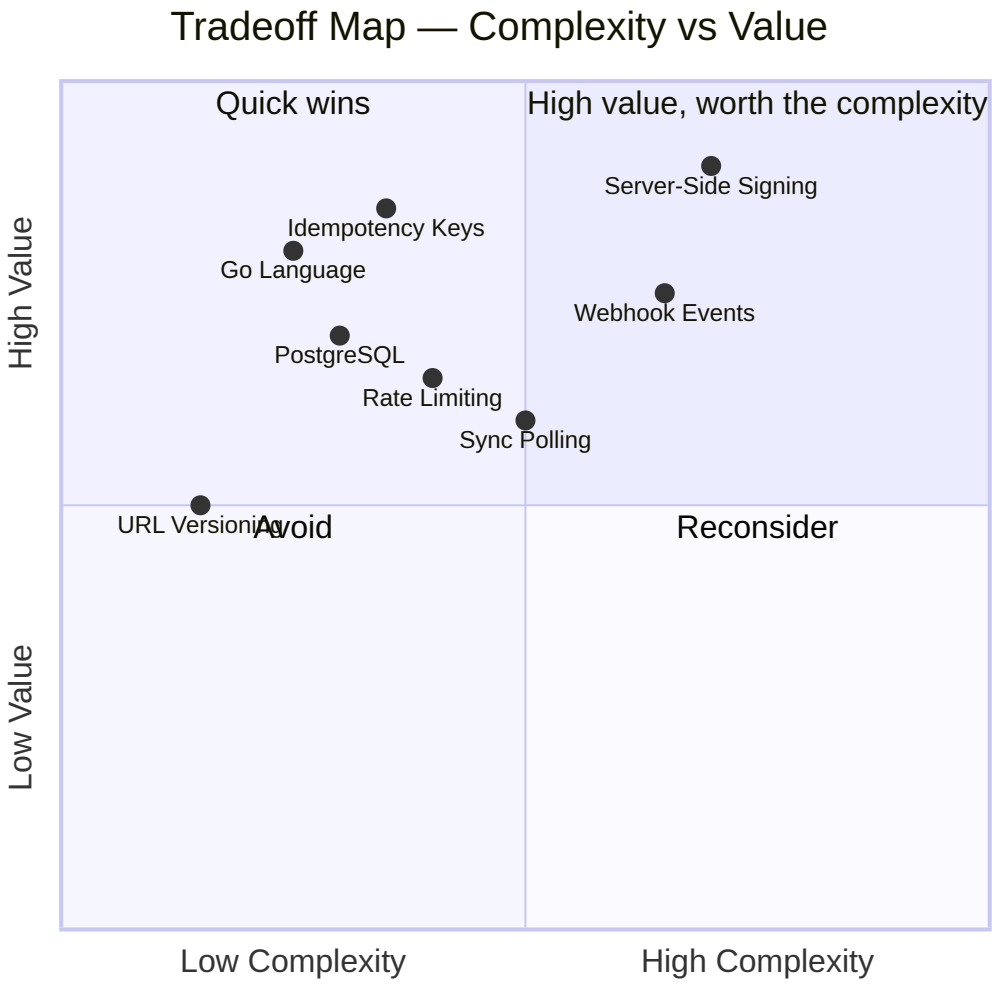
Aspect	Detail
Decision	URL-based versioning (`/api/v1/ ...`) rather than header-based versioning

Aspect	Detail
Rationale	Explicit, easy to route at the load-balancer level, simple to document and test
Tradeoff	URL pollution; harder to share common paths across versions
Mitigation	Internal handler code is version-agnostic; only the router layer maps versions to handlers

Decision 8: Webhook-Based Event Delivery

Aspect	Detail
Decision	Events are delivered via outbound webhooks (push model) rather than requiring clients to poll
Rationale	Real-time delivery; reduces client complexity; aligns with modern event-driven architectures
Tradeoff	Requires reliable outbound HTTP; must handle receiver downtime with retries; HMAC signing for security adds complexity
Mitigation	Exponential backoff retries (3 attempts); delivery status dashboard; dead-letter logging after max retries

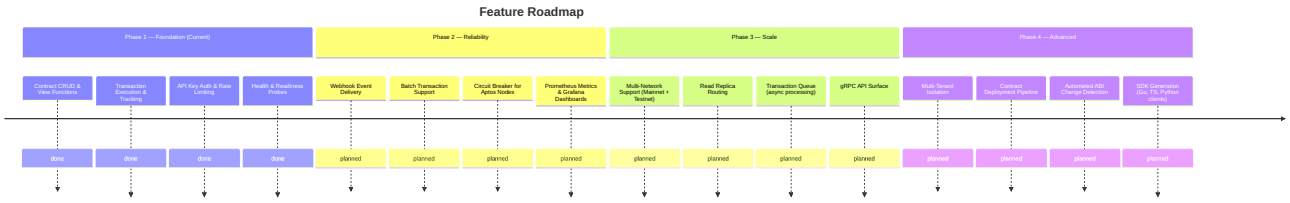
Decision Summary Matrix



---

13. Future Expansions

13.1 Roadmap Overview



13.2 Detailed Expansion Plans

Multi-Network Support

- Support simultaneous connections to Mainnet, Testnet, and Devnet

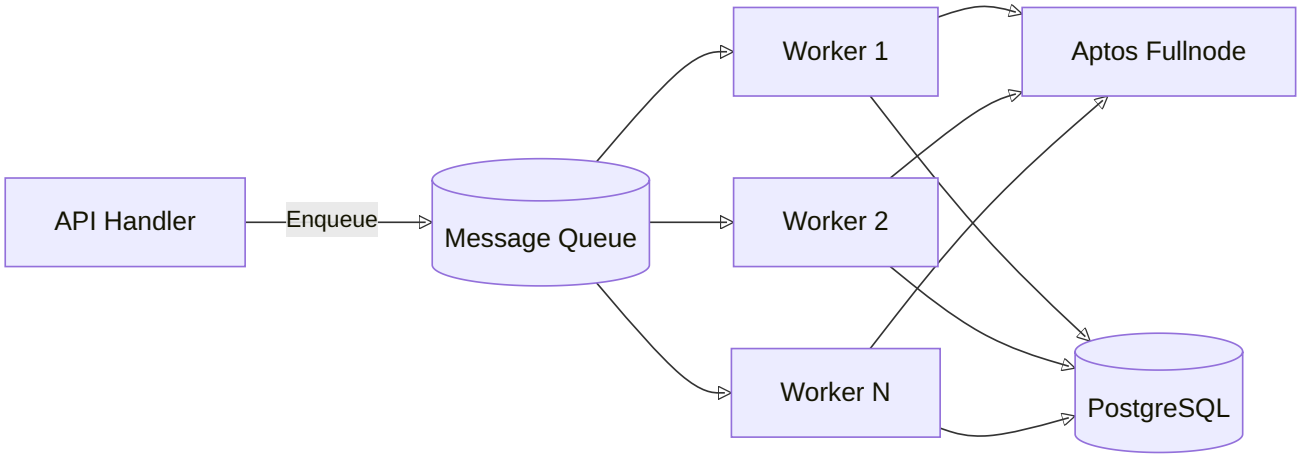
- Network selection per request via header or contract-level default
- Independent rate limits and connection pools per network

gRPC API Surface

- Add a gRPC interface alongside REST for internal service-to-service communication
- Proto definitions generated from the same domain models
- Bidirectional streaming for real-time transaction status updates

Transaction Queue / Async Processing

- Replace synchronous polling with a message queue (NATS / RabbitMQ)
- Workers consume from the queue, submit to chain, and update status
- Enables higher throughput and better backpressure handling



Contract Deployment Pipeline

- Upload Move source code via API
- Compile, test, and deploy through a managed pipeline
- Version tracking and rollback support

Automated ABI Change Detection

- Periodic polling of registered contract ABIs
- Diff detection with alerting when a module's ABI changes on-chain
- Breaking change notifications to webhook subscribers

SDK Generation

- OpenAPI 3.1 spec auto-generated from handler annotations
- Client SDKs generated for Go, TypeScript, and Python

- Published to package registries with version alignment

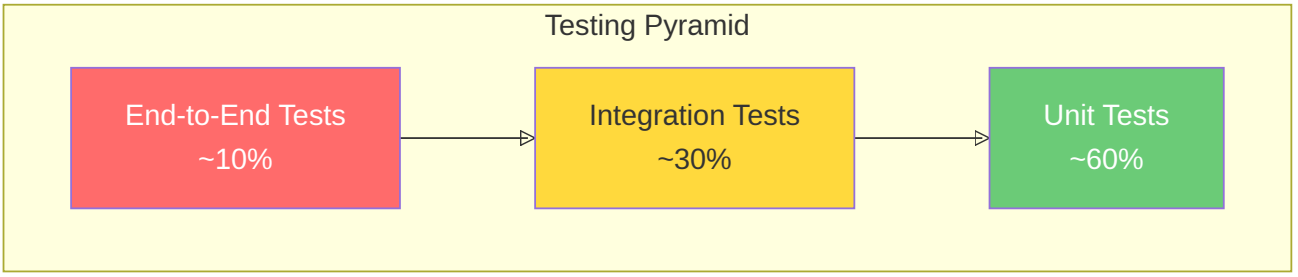
Multi-Tenant Isolation

- Organization-level namespacing for contracts and accounts
- Resource quotas per tenant
- Billing integration for metered usage

---

14. Future Testing & Integration Plans

14.1 Testing Pyramid

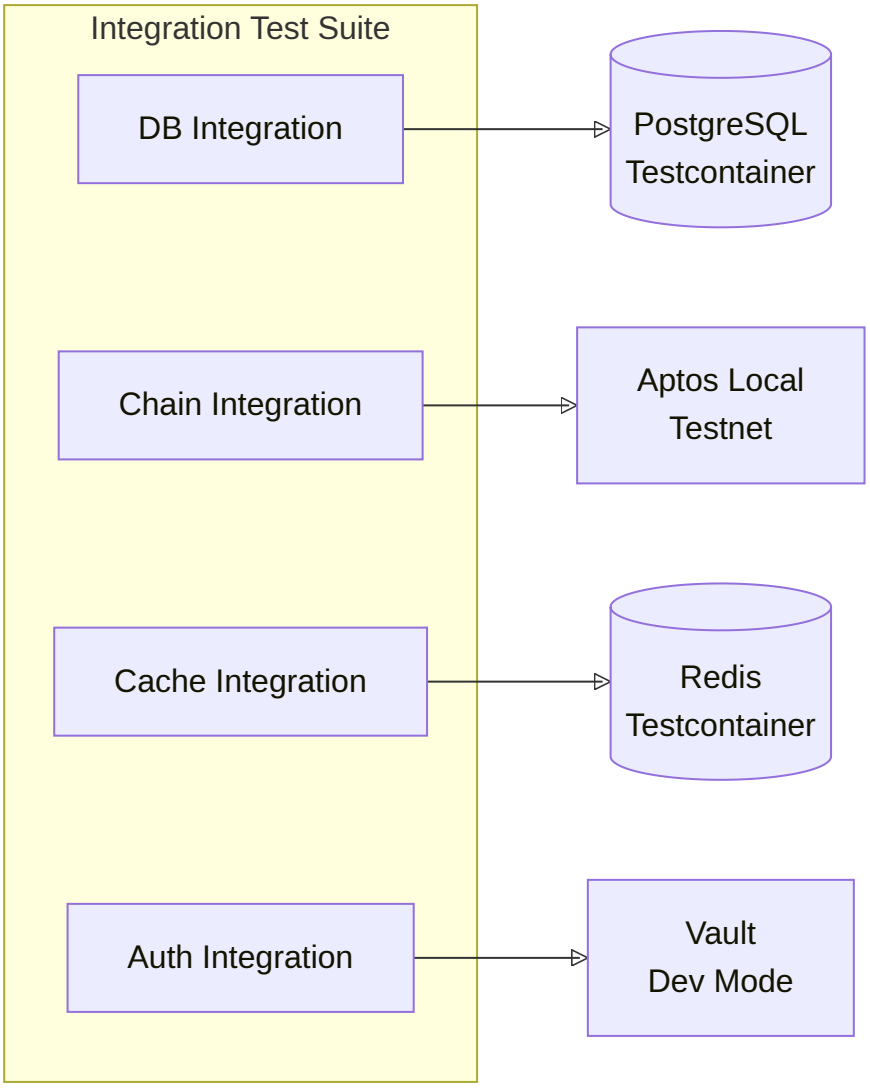


14.2 Unit Testing Plan

Component	Strategy	Tools
Handlers	Test input validation, response mapping, error paths	`net/http/httptest`, `testify`
Services	Mock repository + chain adapter; test business logic in isolation	`testify/mock`, `gomock`
Repository	Test query building and mapping; mock DB connection	`sqlmock`, `testify`
Chain Adapter	Mock Aptos HTTP responses; test BCS encoding, retries	`httptest`, `testify`
Middleware	Test each middleware in isolation (rate limiter, auth, CORS)	`httptest`
Models	Validation rules, JSON marshaling/unmarshaling	`testify/assert`

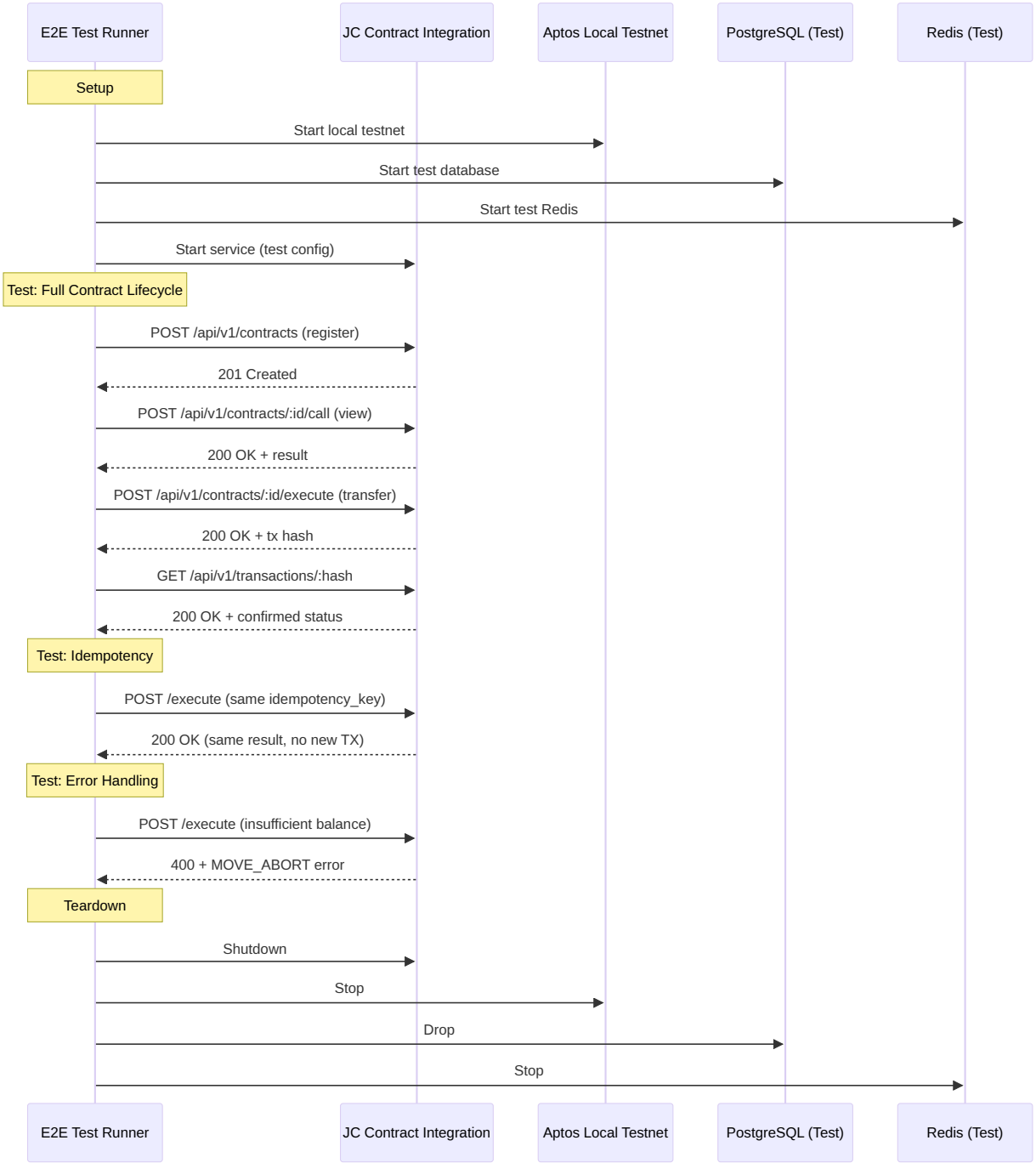
Target: ≥ 80% code coverage across all packages.

14.3 Integration Testing Plan

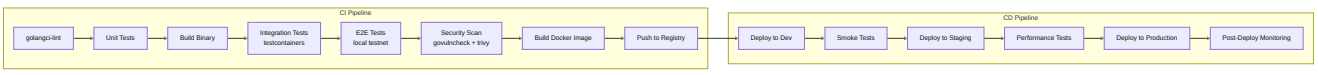


Test Category	Scope	Infrastructure
Database Integration	Full CRUD lifecycle, migrations, constraints, concurrent access	PostgreSQL via testcontainers-go
Chain Integration	Contract call, execute, simulate against a real node	Aptos local testnet (`aptos node run-local-testnet`)
Cache Integration	Rate limit counting, cache invalidation, TTL behavior	Redis via testcontainers
Auth Integration	Full API key lifecycle: create → authenticate → scope check → expire	In-memory or testcontainer DB
Webhook Integration	Event detection → webhook dispatch → retry on failure	Mock HTTP server + testcontainer PostgreSQL

14.4 End-to-End Testing Plan



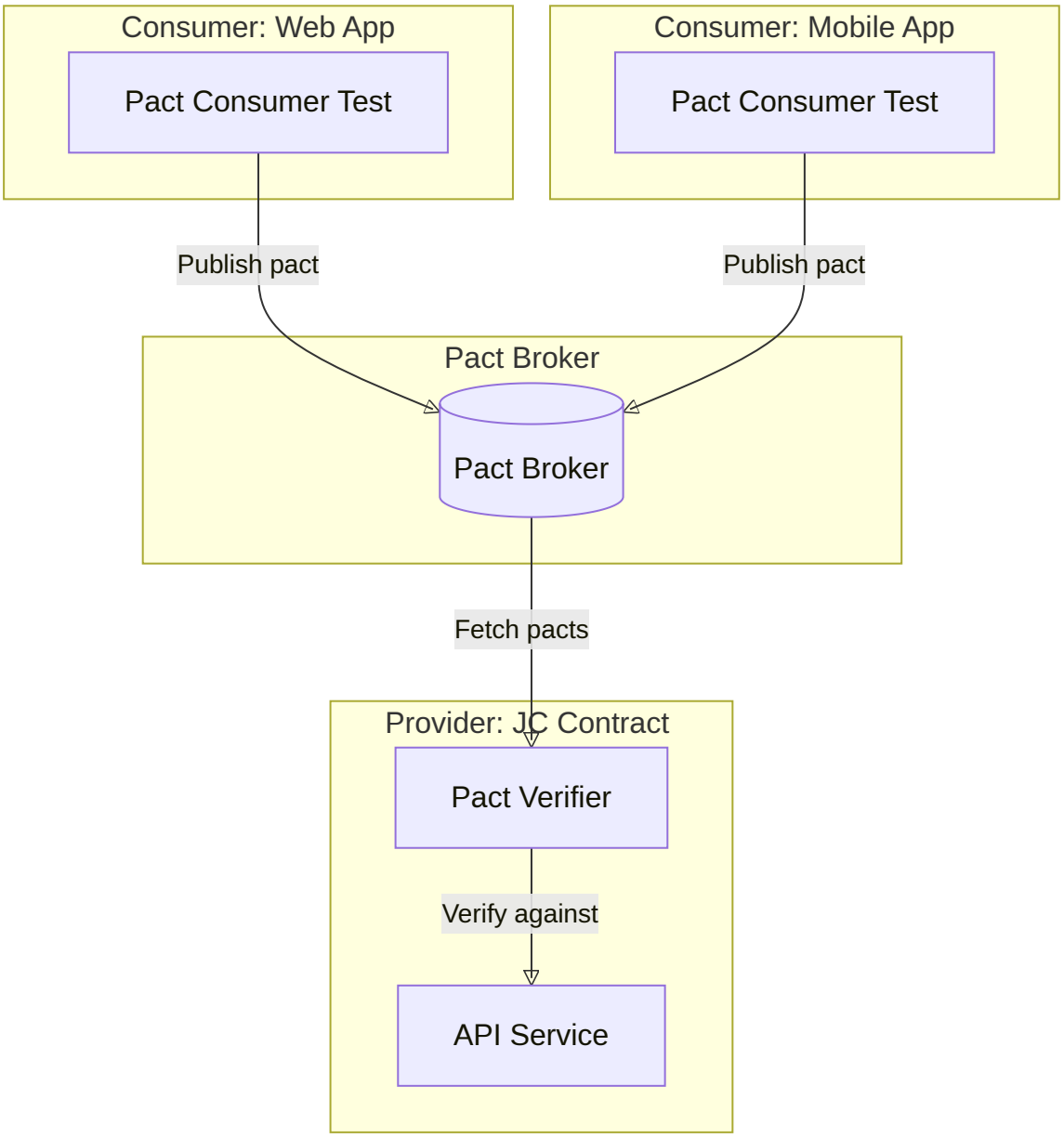
14.5 CI/CD Pipeline



14.6 Performance & Load Testing

Test Type	Tool	Target
Load Testing	k6 / Vegeta	Sustain 1,000 RPS on read endpoints; 100 RPS on execute
Soak Testing	k6 (extended)	24-hour run at 50% capacity; monitor for memory leaks
Spike Testing	k6	10x traffic burst for 60s; verify graceful degradation
Latency Profiling	pprof + Jaeger	P99 < 200ms for reads; P99 < 2s for execute (excl. chain time)

14.7 Contract Testing (Consumer-Driven)



- Consumer teams publish Pact contracts defining expected request/response pairs

- Provider CI verifies all consumer pacts before deployment
- Breaking changes are caught before they reach production

14.8 Security Testing

Area	Approach
Dependency Scanning	`govulncheck` on every CI run; Dependabot for automated PRs
SAST	`gosec` for Go-specific security issues
Container Scanning	Trivy on Docker images
API Fuzzing	RESTler or custom fuzzer against OpenAPI spec
Penetration Testing	Annual third-party pen test on staging environment
Secret Detection	`gitleaks` pre-commit hook + CI check

---

15. Appendix

A. Environment Variables

Variable	Required	Default	Description
`PORT`	No	`8080`	HTTP listen port
`ENV`	No	`development`	Environment name
`DATABASE_URL`	Yes	—	PostgreSQL connection string
`REDIS_URL`	Yes	—	Redis connection string
`APTOS_NODE_URL`	Yes	—	Fullnode REST API URL
`APTOS_INDEXER_URL`	No	—	Indexer GraphQL URL
`APTOS_NETWORK`	No	`testnet`	Network name
`VAULT_ADDR`	Yes (prod)	—	HashiCorp Vault address
`VAULT_TOKEN`	Yes (prod)	—	Vault authentication token
`LOG_LEVEL`	No	`info`	Logging level
`RATE_LIMIT_READ`	No	`100`	Reads per minute per key

Variable	Required	Default	Description
`RATE_LIMIT_WRITE`	No	`20`	Writes per minute per key
`TX_POLL_TIMEOUT`	No	`10s`	Max time to poll for TX confirmation
`TX_POLL_INTERVAL`	No	`500ms`	Polling interval for TX status

B. HTTP Status Code Usage

Status	When Used
`200 OK`	Successful read, view function call, or confirmed transaction
`201 Created`	Resource created (contract registered, account created)
`202 Accepted`	Transaction submitted but not yet confirmed
`204 No Content`	Successful deletion
`400 Bad Request`	Validation error in request body/params
`401 Unauthorized`	Missing or invalid API key
`403 Forbidden`	Valid API key but insufficient scope
`404 Not Found`	Resource does not exist
`409 Conflict`	Duplicate resource (e.g., contract already registered)
`422 Unprocessable Entity`	Semantically invalid (e.g., invalid Move address format)
`429 Too Many Requests`	Rate limit exceeded
`500 Internal Server Error`	Unexpected server failure
`502 Bad Gateway`	Aptos node returned an error
`503 Service Unavailable`	Service is starting up or Aptos node is unreachable

C. Glossary

Term	Definition
Move	The smart contract language used by the Aptos blockchain
BCS	Binary Canonical Serialization — Aptos transaction encoding format
Fullnode	An Aptos node that serves the REST API and stores the full blockchain state

Term	Definition
Indexer	Aptos service that provides aggregated/enriched blockchain data via GraphQL
Sequence Number	Per-account transaction counter used to prevent replay attacks
View Function	A read-only Move function that does not modify state
Entry Function	A Move function that can be called via a transaction to modify state
ABI	Application Binary Interface — describes the functions and types of a Move module
Gas	Computational cost unit for executing transactions on Aptos
HSM	Hardware Security Module — physical device for secure key storage

---

Document generated for `aptos-labs/jc-contract-integration` — Last updated 2026-02-27